

Joint Activation and Structured Sparsity for ECG Arrhythmia Classification on Akida 1

Akida 1 Model Zoo

August 2026

Abstract

This is the fourth in a series of repeated studies on whether combining activation-sparsity regularization with structured (channel) pruning costs more, less, or the same as the sum of each mechanism’s individual accuracy cost. The first three studies (Visual Wake Words, PlantVillage, Speech Commands) each found a different pattern. This report, on ECG arrhythmia classification from single-lead MIT-BIH recordings (a 1D-signal-to-scalogram task, this series’ first non-image, non-audio modality), finds a fourth: activation regularization actively *improves* accuracy here rather than merely costing less than expected, and the aggressive joint configuration (74.6% activation sparsity, a 25.7% parameter reduction) *strictly dominates* a lighter alternative — better accuracy, more sparsity, and a larger parameter cut, simultaneously, with no tradeoff to make. This is consistent with, and explained by, the companion single-axis experiment’s own finding that no regularizer strength tested on this model ever caused the accuracy collapse seen in the other three studies, plausibly because this project’s class-weighted loss (addressing severe class imbalance) makes the "collapse to the majority class" failure mode far less attractive here.

Contents

1	Introduction	2
2	Background	2
2.1	Akida 1 and activation sparsity	2
2.2	ECG arrhythmia classification	3
3	Dataset	3
4	Model architecture	3
5	Methodology	4
5.1	Activation sparsity: normalized Hoyer-Square regularization	4
5.2	Structured sparsity: Network Slimming, located positionally	4
5.3	Float-only evaluation	5
5.4	Training pipeline	5
6	Experimental design	5

7	Results	6
7.1	Activation regularization improves accuracy here, not just sparsity	7
7.2	The joint configuration also beats baseline	7
7.3	A fourth distinct pattern: the aggressive configuration strictly dominates the light one	8
7.4	Consistent with this project’s own collapse-free single-axis behavior	8
8	Discussion and limitations	8
9	Conclusion	8

1 Introduction

This is the fourth of four repeated studies investigating the interaction between activation-sparsity regularization and structured channel pruning. The first three reached three different conclusions:

- **VWW**: the joint configuration was complementary (sub-additive) throughout, and a lighter configuration simply traded some sparsity/size benefit for a smaller accuracy cost along a smooth curve.
- **PlantVillage**: an aggressive joint configuration was sharply antagonistic (a pruning-recovery problem), but a lighter configuration avoided this almost entirely and was a strict win on every axis.
- **Speech Commands**: the sign of the interaction flipped with regularization strength (complementary when aggressive, mildly antagonistic when light), and neither operating point dominated the other.

This report asks the same two questions — does the joint cost more or less than the sum of its parts, and can a better operating point be found — on ECG arrhythmia classification (3-class beat classification from continuous-wavelet-transform scalograms of single-lead ECG, MIT-BIH Arrhythmia Database), a fourth model, dataset, and signal modality (1D physiological time series, rather than vision or audio).

All work in this report is on the branch `arrhythmia_classification-joint`. This follows up on two prior efforts: the single-axis activation-sparsity experiment on branch `akida1-sparsify-arrhythmia` (`SPARSITY_EXPERIMENT.md`), and the joint VWW, PlantVillage, and Speech Commands studies.

2 Background

2.1 Akida 1 and activation sparsity

Akida 1 accelerates inference by skipping computation on zero-valued activations. Structured (channel) pruning is a separate property – how much of the model’s parameter count has been physically removed – that Akida 1’s execution model does not exploit for speed; its value is reduced memory footprint.

2.2 ECG arrhythmia classification

The task is 3-class heartbeat classification (Normal, Supraventricular, Ventricular) from single-lead ECG recordings in the MIT-BIH Arrhythmia Database. Each heartbeat is extracted around its annotated R-peak, transformed into a continuous-wavelet-transform (CWT) scalogram image, and augmented with four rows encoding RR-interval timing features. Unlike the other three studies (two image tasks, one audio task), the raw signal here is a 1D physiological time series, and the model consumes a derived time-frequency image representation of it rather than a directly-sampled image or audio spectrogram.

3 Dataset

Table 1: Dataset summary, as prepared by `scripts/data.py`’s `ECGDatasetBuilder`.

Property	Value
Source	MIT-BIH Arrhythmia Database (PhysioNet), 48 records
Classes	3 (N: normal, S: supraventricular, V: ventricular ectopic)
Class weights	{0 : 1.0, 1 : 6.0, 2 : 3.0} (addresses severe class imbalance)
Beat window	250 samples before/after the R-peak, resampled to 360
Feature transform	CWT scalogram (32×32) + 4 RR-interval rows appended
Input shape	$36 \times 32 \times 1$
Train / validation / test	40,447 / 10,112 / 49,280 beats (stratified 80/20 train/val split)

The RR-interval rows are z-scored using statistics computed from the training split only, then stacked beneath the min-max-normalized scalogram, giving each sample a real dynamic range of approximately $[-5.59, 14.83]$ – substantially wider and more asymmetric than a naively-assumed $[-1, 1]$ or $[0, 1]$ range. (The companion single-axis experiment found and fixed a pre-existing bug where `scripts/eval.py` assumed the latter when converting to Akida, silently corrupting hardware-accuracy measurements; not relevant to this float-only study; see `SPARSITY_EXPERIMENT.md` and `scripts/input_scaling.py`.)

4 Model architecture

The model (`build_akida_model` in `scripts/model.py`) is a small, custom Akida-targeted CNN: a stem convolution, two depthwise-separable blocks with max-pooling, a final depthwise-separable block without pooling or an immediately-following activation, global average pooling, and a two-layer dense classifier head. At 23,363 parameters it is the smallest of the four models studied across this series (VWW: 226,906; PlantVillage: 1,156,054; Speech Commands: 22,668 – comparable in scale to Speech Commands, both far smaller than the two vision examples). Like the other three, it is a strict feed-forward chain with no residual/skip connections.

Table 2: Model architecture. Every `*_sepconv/stem_conv` row is immediately followed by its own BatchNormalization (omitted for brevity; parameter counts include them). `block3_bn` is notably *not* followed by an activation – the ReLU is deferred until after global average pooling.

Layer	Type	Output shape	Params (layer + BN)
<code>input</code>	Input	$36 \times 32 \times 1$	0
<code>stem_conv</code>	Conv2D	$36 \times 32 \times 16$	464
<code>block1_sepconv</code>	SeparableConv2D	$36 \times 32 \times 32$	1,040
	MaxPooling2D	$18 \times 16 \times 32$	0
<code>block2_sepconv</code>	SeparableConv2D	$18 \times 16 \times 64$	3,104
	MaxPooling2D	$9 \times 8 \times 64$	0
<code>block3_sepconv</code>	SeparableConv2D (no activation)	$9 \times 8 \times 128$	10,304
	GlobalAveragePooling2D	128	0
	ReLU	128	0
<code>dense</code>	Dense (linear)	64	8,256
	ReLU	64	0
<code>dense_1</code>	Dense (linear, logits)	3	195
Total			23,363
Trainable / non-trainable			22,883 / 480

The classifier head reads from `block3_sepconv`’s output only after global average pooling and an intervening ReLU, structurally similar to the other three examples’ final-feature-layer-to-classifier connection, and for the same reason: `block3_sepconv` is excluded from the prunable set (Section 5.2).

5 Methodology

The two sparsity mechanisms and the float-only rationale are identical to the prior three studies; this section highlights what differs for this project.

5.1 Activation sparsity: normalized Hoyer-Square regularization

The established best point from the single-axis experiment is normalized Hoyer-Square at $\lambda = 3$: 68.2% activation sparsity (Akida-measured) for only a ~ 0.9 -point accuracy cost from the unregularized baseline – the cheapest sparsity gain of any of the four model-zoo examples studied. Uniquely among the four, this project’s single-axis sweep found **no regularizer strength that caused a training collapse**, even at 30–100 $\times$ the strengths that fully destroyed VWW’s, PlantVillage’s, and Speech Commands’ accuracy – attributed in `SPARSITY_EXPERIMENT.md` to the class-weighted loss (Table 1) making "predict only the majority class" a much less attractive failure mode here.

5.2 Structured sparsity: Network Slimming, located positionally

Network Slimming [1] is applied via `scripts/arrhythmia_structured_sparsity.py`, adapted from the VWW implementation with one change: this project’s BatchNormalization layers are named `{block}_bn` (e.g. `block1_bn`) rather than `{layer}/BN`. Rather than hardcoding yet another naming convention, the paired BN is now located *positionally* – the layer immediately following the target in `model.layers` – a more robust approach in general, since it makes no assumption

about naming at all. **Pruning scope:** `block1_sepconv` and `block2_sepconv` (2 of this model’s 3 SeparableConv2D blocks); `block3_sepconv` is excluded since it feeds the classifier head directly. This model has only 3 SeparableConv2D blocks total (versus VWW’s and PlantVillage’s 10, Speech Commands’ 4), so its prunable set is smaller by construction than any of the other three.

5.3 Float-only evaluation

As with the other three studies, no `cnv2snn quantize/QAT/convert` is used anywhere in this pipeline. Activation sparsity is the mean fraction of exactly-zero ReLU outputs on 1,000 held-out samples, and is **not directly comparable** to the single-axis experiment’s Akida-measured figures for the same reason as the other three studies.

5.4 Training pipeline

Unlike VWW, PlantVillage, and Speech Commands – each of which exposes a reusable `train_*(model, ...)` function that this series’ sweep scripts import directly – this project’s `scripts/train.py` runs float training, quantization, and QAT all inline in a single `__main__` script, with no importable training function. This report’s sweep instead calls `build_akida_model/apply_activity_regularizer` directly and implements its own simple training loop (float-only, so no quantize/QAT stage is needed regardless).

Best-checkpoint reload. `train.py`’s own convention is `ModelCheckpoint(monitor="val_loss", save_best_only=True)` together with `EarlyStopping(patience=8, restore_best_weights=False)` – meaning the live in-memory model immediately after `fit()` is *not* necessarily the best one seen during training. This report’s training loop reproduces that exactly, reloading the checkpointed best-val-loss model after every training phase.

Phase 1 (float training, up to 80 epochs). Matches `train.py`’s float-training scale (Adam, learning rate 3×10^{-3} , `ReduceLROnPlateau` and `EarlyStopping(patience=8)` – so 80 is a ceiling, not a fixed count).

Pruning (if $p > 0$). Channels pruned per Section 5.2.

Phase 2 (post-prune fine-tuning, up to 40 epochs, learning rate 3×10^{-4}). Only run when $p > 0$.

The class-weighted loss {0:1.0, 1:6.0, 2:3.0} (pre-existing, addressing this dataset’s severe class imbalance) is applied unchanged in every phase.

Training speed. At 23,363 parameters and small $36 \times 32 \times 1$ inputs, each epoch takes roughly 5–7 seconds on GPU 1 of the shared training server. Despite the two-phase-with-fine-tuning structure, the full pilot (4 configurations) and follow-up (3 more) together completed in well under 20 minutes.

6 Experimental design

Pilot (2×2 grid). All four combinations of activation regularization on/off ($\lambda = 3$, the established best single-axis point) and pruning on/off ($p = 40\%$, a starting guess mirroring the other three studies’ choice on this axis).

Follow-up (lighter settings). Three lighter points, halving both knobs: $\lambda = 1.5$ and $p = 20\%$, individually and jointly.

Table 3: All configurations run. λ is the normalized Hoyer-Square strength; p is the structured prune target.

Tag	λ	p
baseline	0	0%
activation_only	3	0%
structured_only	0	40%
joint	3	40%
activation_light	1.5	0%
structured_light	0	20%
joint_light	1.5	20%

7 Results

Table 4: Full results. Accuracy drop is relative to **baseline** (93.85%); a *negative* drop means the configuration outperformed baseline. Parameter reduction is relative to **baseline**’s 23,363 parameters. Activation sparsity is the float-model measurement and is *not* directly comparable to the single-axis experiment’s Akida-measured numbers.

Tag	λ	p	Accuracy	Acc. drop	Act. sparsity	Params (reduction)
baseline	0	0%	93.85%	–	48.7%	23,363 (–)
activation_only	3	0%	95.26%	–1.41pt	79.2%	23,363 (0%)
activation_light	1.5	0%	93.73%	0.12pt	73.0%	23,363 (0%)
structured_only	0	40%	93.44%	0.41pt	45.5%	17,370 (25.7%)
structured_light	0	20%	93.35%	0.50pt	47.3%	20,330 (13.0%)
joint	3	40%	94.43%	–0.57pt	74.6%	17,370 (25.7%)
joint_light	1.5	20%	94.01%	–0.15pt	70.8%	20,330 (13.0%)

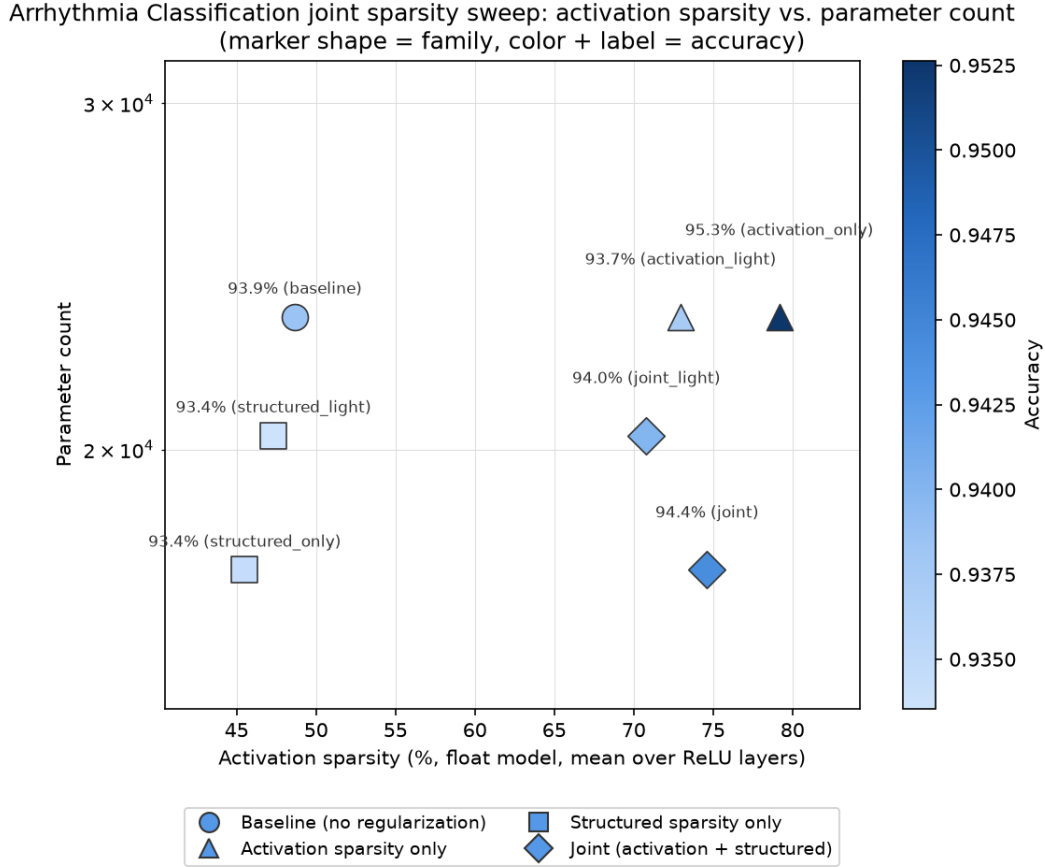


Figure 1: Activation sparsity vs. parameter count for all seven configurations. Marker shape encodes family (circle: baseline, triangle: activation sparsity only, square: structured sparsity only, diamond: joint); marker color and the accompanying label encode accuracy.

7.1 Activation regularization improves accuracy here, not just sparsity

`activation_only` beats `baseline` by 1.41 accuracy points (93.85% $\rightarrow$ 95.26%) while simultaneously gaining 30.5 points of activation sparsity. This matches the single-axis experiment’s own finding of non-monotonic, sometimes-*improving* accuracy as a function of regularizer strength on this dataset, attributed there to the class-weighted loss making mild regularization act as a generalization aid rather than a pure capacity reduction. None of VWW, PlantVillage, or Speech Commands showed this behavior – every activation regularizer tested on those three cost at least some accuracy.

7.2 The joint configuration also beats baseline

`joint` reaches 94.43% accuracy (a 0.57-point *gain* over baseline) with 74.6% activation sparsity and the full 25.7% parameter reduction `structured_only` achieves alone. Structured pruning’s own small cost (`structured_only`: 0.41pt) is more than offset when combined with activation regularization’s accuracy gain.

7.3 A fourth distinct pattern: the aggressive configuration strictly dominates the light one

Unlike all three prior studies – where the lighter joint configuration either simply cost less (VWW), was a strict win on every axis (PlantVillage), or traded one axis for another with neither point dominating (Speech Commands) – here `joint` beats `joint_light` on *every* axis simultaneously: higher accuracy (94.43% vs. 94.01%), higher activation sparsity (74.6% vs. 70.8%), and a larger parameter reduction (25.7% vs. 13.0%). There is no tradeoff to make between these two points; pushing both regularization knobs harder simply helped, within the range tested.

7.4 Consistent with this project’s own collapse-free single-axis behavior

This finding is not an isolated anomaly: it is consistent with `SPARSITY_EXPERIMENT.md`’s own observation that no regularizer type or strength tested on this model ever produced the accuracy collapse the other three projects’ sweeps all eventually hit, even at 30–100× the strength that destroyed those models. If stronger regularization/pruning simply keeps paying off up to some much higher strength than tested here, that would explain why the aggressive joint corner in this pilot has not yet found a downside.

8 Discussion and limitations

Has the ceiling been found? Since `joint` dominates `joint_light` rather than trading against it, and given the single-axis experiment’s own note that this model’s collapse point (if one exists) lies well beyond anything tested so far, it is plausible that pushing λ and/or p even higher than this pilot’s “aggressive” corner would continue to help. A follow-up sweep at, e.g., $\lambda = 10$ and $p = 60\%$ would test whether the dominance pattern holds further out or eventually reverses.

This dataset’s noise floor warrants caution. `SPARSITY_EXPERIMENT.md` documented non-monotonic, seed-sensitive results throughout its own sweep on this small, heavily-imbalanced dataset (49,280 test beats, but very few training examples for the minority S and V classes). The accuracy *gains* reported here are plausible and mechanistically explicable, but have not been confirmed across multiple random seeds – a single training run’s ranking between closely-spaced configurations should not yet be treated as a firm conclusion.

Float-vs-hardware sparsity gap and no latency validation. As with the other three studies, the float-model sparsity measurements here are not on the same footing as the single-axis experiment’s Akida-hardware/software-simulator-measured figures, and this report makes no claim about actual Akida 1 latency or power.

9 Conclusion

Repeating the joint activation-sparsity and structured-sparsity investigation a fourth time, on ECG arrhythmia classification, produced yet another distinct pattern from VWW, PlantVillage, and Speech Commands. Activation regularization measurably *improves* accuracy on this dataset rather than merely costing less than expected, and the aggressive joint configuration ($\lambda = 3$, 40% pruning) strictly dominates a lighter alternative on every axis tested – accuracy, activation sparsity, and parameter count all improve together, with no tradeoff to navigate. This traces to a property

specific to this dataset and its class-weighted training loss: unlike the other three examples, no regularizer strength tested here has ever caused a training collapse, so pushing harder within the tested range simply keeps paying off. Across all four studies now completed, the overarching lesson holds: the interaction between activation-sparsity regularization and structured pruning is model- and dataset-specific in ways that range from complementary to antagonistic to (as seen here) unambiguously beneficial, and must be measured directly for each new model rather than assumed from any prior result.

References

- [1] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang. Learning Efficient Convolutional Networks through Network Slimming. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.
- [2] H. Yang, W. Wen, and H. Li. DeepHoyer: Learning Sparser Neural Network with Differentiable Scale-Invariant Sparsity Measures. In *International Conference on Learning Representations (ICLR)*, 2020.
- [3] G. B. Moody and R. G. Mark. The Impact of the MIT-BIH Arrhythmia Database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3):45–50, 2001.